ARCHITECTURE CASE STUDY

StudyHub

A local-first, AI-assisted study platform: the product, its architecture, and the engineering decisions behind it

Product version 4.7.0 · 2026-11-10
A living document, generated from the code

Document control

Item	Detail
Subject	StudyHub, a plugin family for DeepSeek Harness (DSH), at product version 4.7.0 (2026-11-10).
Purpose	To describe what the product is, how it is built, why it is built that way, what it costs and risks, and where it should go next, in one document that engineers, architects and evaluators can use.
Status	Living document. It describes the code as it is. The repository is the source of truth.
How the numbers are produced	A limit, default or count that a constant or table in the code defines is read from the code when this document is built, and a test compares it with a committed snapshot. Measurements are quoted from the project's own performance records with their date. Opinions are labelled as assessment.
How to update	See docs/case-study/README.md in the repository: rebuild after changing a quoted limit, default or count; edit the prose when a flow, a module or a decision changes.

Contents

1. Executive summary
2. Product overview
3. System context and capabilities
4. Requirements and quality attributes
5. Architecture overview
6. Key design decisions
7. Critical flows
8. Data architecture and the performance engineering story
9. Runtime, jobs and recovery
10. AI architecture: cost, trust and failure
11. Packaging, delivery and quality engineering
12. Security and privacy analysis
13. Scalability analysis and a scale-out scenario
14. Assessment: strengths, weaknesses, risks and recommended next steps

Appendix A: Glossary
Appendix B: Sources

1. Executive summary

The product. StudyHub turns a learner's own lecture material into practice questions that cite their source page, schedules reviews, and shows progress, all inside DeepSeek Harness and on the learner's own computer. It serves one learner per library, is local-first, and is bill-by-use: the learner pays a model provider directly and sees an estimate before each run.

What shaped the architecture. Four forces dominate. (1) *Trust*: a generated question is only worth practising if it is grounded in the source and checked, so grounding is done in code, review is done in a separate call, and the learner always reads a draft. (2) *Data safety*: the library is a semester of work, so every write is atomic, versioned and idempotent, and a backup is taken before any migration. (3) *Cost and latency*: model calls are the expensive, slow, failure-prone step, so everything else (reading, practising, scheduling) is free of them, and every AI job is budgeted, cancellable and checkpointed. (4) *Host coupling*: StudyHub is a guest in DSH, which supplies models, keys, tools and the window, and which limits what the panel may do.

The solution in one paragraph. A modular monolith per library. One `StudyRuntime` hosts 19 contexts with versioned APIs, declared dependencies and operation grants, enforced by lint and tests. A local, content-addressed shard store replaces its manifest atomically. One contract describes every background job. Commits are idempotent and guarded by version checks and receipts. A React panel polls a fingerprinted snapshot. External services (models, transcription, Gemini Live, MinerU, retrieval) sit behind narrow ports.

What the evidence says. The 2.4.0 performance work took a real 1,000-source library from an 8.8 MB snapshot and about 500 ms per click-and-refresh to 2 MB and about 100 ms, and host memory after 100 polls from 700 MB to 275 MB (section 8). The generation pipeline reports its failures by cause, and a stopped or interrupted run keeps every approved question (sections 7 and 10).

Recommended next steps (section 14): bound the in-memory library before libraries grow another order of magnitude; replace polling with change notification where the host allows it; finish the job-runtime migration so that one runtime owns every job; and keep the feature surface understandable for newcomers.

2. Product overview

2.1 The problem

University students learn from lecture slides, PDFs, notes and recorded lectures, in English or Chinese. Turning that material into practice is slow, and the tools that exist share some inconveniences:

- Writing flashcards and questions by hand from slides takes longer than studying them.
- Questions from a general chat assistant are often not tied to the lecture, may be wrong, and cannot be traced to a page.
- The cost of AI help is unpredictable: every call is billed by a model provider, often without an estimate beforehand.
- Lecture material and recordings are private, and many learners do not want to upload them to yet another service.
- Spaced repetition, mistakes, exams and reading live in different tools that know nothing about each other.

AI-assisted study platforms also face challenges of their own: trust in generated content, protection of private material, honest cost, the robustness of long-running AI jobs (timeouts, quotas, restarts, cancellation), working with whichever model the learner chooses, and staying understandable to a newcomer who meets a large feature set.

2.2 The product

StudyHub is a free, open-source product (MIT) that turns the learner’s own sources into questions that each link back to their page, then helps the learner practise, review and see progress. It is a plugin family for DeepSeek Harness (DSH), an AI agent application that runs either as a desktop app or as a web page served from the learner’s own computer, and StudyHub opens inside it. The learner’s library is a folder on the learner’s computer, model keys are held by DSH, and nothing is sent to a StudyHub server because there is none.

2.3 Scope and release history

The capabilities fall into two groups (section 3): the **core study loop** (sources, grounded generation with independent review, practice with spaced repetition, weak points) and **capture and guidance** (audio and live class, large-PDF conversion, retrieval, the review book, learning flows, daily planning and guidance for newcomers). The repository has 81 changelog entries between 2026-10-01 and 2026-11-10, and the product is at version 4.7.0.

2.4 Extensibility as a product requirement

New capabilities must be addable without changing existing ones, and third parties must be able to extend StudyHub through the same APIs. StudyHub is therefore a set of separately installable plugins (the workbench and 6 capability plugins: runtime, materials, bank, study, generation, audio) that share one `studyRuntime` service. A third party can register its own context with a versioned API, owned collections and declared dependencies, register typed tools and UI contributions, and run background work through the same task service. Unloading the last owner of a context removes it and its tools; the durable records stay.

3. System context and capabilities

3.1 Actors and external systems

1. **Learner**: the person studying. Works in the StudyHub panel and owns the library.
2. **DSH host**: the desktop app or local web server in which StudyHub runs. It provides the plugin manager, sessions, the tool registry, the sidebar and previews, and it holds the model keys and routes.
3. **Main chat assistant**: DSH's assistant. It may call StudyHub's tools to read or change the library on the learner's request.
4. **Model provider**: the language-model service the learner configured in DSH. StudyHub reaches it only through the host's route.
5. **Local file system**: where the library (shards) and the retained original files live.
6. **GitHub Releases**: where StudyHub is published and where the update check asks for the latest version.
7. **Transcription providers** (SiliconFlow, Groq, Gemini): turn recorded audio into text.
8. **Gemini Live / local SenseVoice**: real-time speech recognition for the live class (cloud via Gemini Live, or on the learner's computer via SenseVoice installed on demand).
9. **MinerU**: a PDF converter that runs locally.
10. **Search extension**: an MCP retrieval server installed from the release.

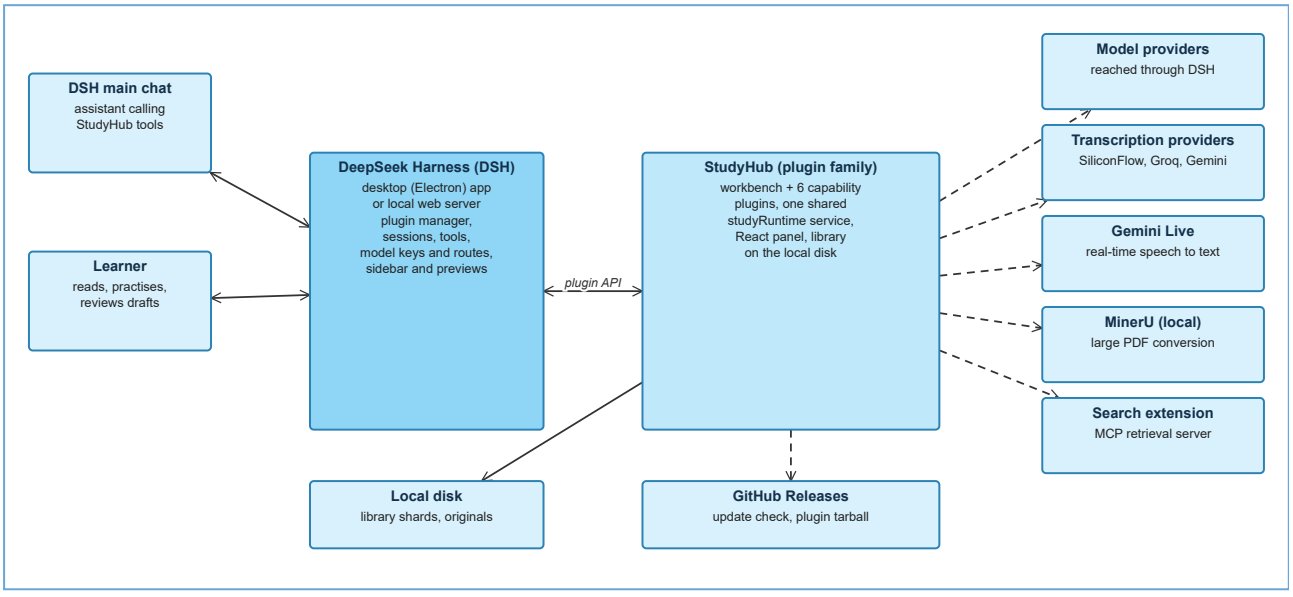


Figure 1: System context

3.2 Core study loop

Capability	Description

Add sources	A learner adds lecture material to a course: several PDF, Word, PowerPoint, Markdown, HTML or text files at once, or pasted text. The platform keeps the original file, projects its text into pages (a slide is a page) and lists the source in the library. Size limits apply per format (for a PDF, 8 MB and 200 pages).
Read sources	A source opens in the reader: the original page, the extracted text, a table of contents and search. Paragraph and table markers lead to the questions and explanations linked to them.
Select and ask	A learner selects a passage and asks about it. The platform first checks that the quote is still where it was (resolved, ambiguous, stale or missing), then answers from the passage and its neighbouring evidence.
Estimate cost	Before a generation run the platform states the expected tokens; no model is called to produce the estimate. After the run it shows the actual tokens.
Generate a deck	The learner chooses sources (by document or chapter), a question type, a count and a difficulty, and starts a background job. The job plans targets with citations, settles the answers, writes the questions and has every part independently reviewed before saving it as a draft.
Review a draft	The learner reads the draft and each question's citation, edits or removes questions, and sees why a question was left out. A draft is kept apart from published questions.
Publish a draft	The platform re-checks every question and publishes the deck in one transaction. A draft that was changed elsewhere in the meantime is refused (stale version) instead of overwritten.
Add to a deck	A selected passage can supplement an existing deck. The approved questions are appended; existing questions, their ids and their review progress stay.
Practise	The learner starts today's study. The platform builds a run from due reviews, weak areas and unfinished work; the learner answers, reveals the explanation and moves on. Answering and scheduling make no model calls.
Rate an answer	A flashcard or open answer is rated from 0 to 5. The platform schedules the next review with FSRS-6 and records the attempt.
Get help	On any question the learner asks a tutor. When the host supports local sub-agents, repeated requests on the same question keep the same tutor for a bounded time.
Find weak points	The platform groups wrong answers by deck or topic, filters them, and can generate variants of the questions the learner got wrong.
Take a mock exam	A multiple-choice paper, a case paper or an oral interview is built from the library, with a report afterwards.
See statistics	Progress, token usage and what still needs work.
Daily recap	After 10 distinct questions in one course on the same day, one recap across its chapters can be generated (optionally automatically).
Manage courses	Courses label sources and decks and hold an exam profile and focus topics. Renaming and merging run as one transaction in the library.
Stop or retry a job	Every background job can be stopped (repeating the request is safe) and, where its kind allows, retried or continued. A stopped job keeps the questions it had already approved.
Back up and restore	The library is exported as one JSON file and restored from it. A restore first saves the current library and is refused while a generation job is active.
Import a JSON deck	Questions the learner already has can be imported from a JSON file in a documented format.
Settings and updates	Question defaults, library location, interface language and component choices are set in Settings. The update check asks GitHub for the latest version and sends no study data.

3.3 Capture and guidance

Capability	Description
Import a recording	A lecture recording (MP3, WAV, M4A, AAC, OGG, FLAC, OPUS, WEBM, AIFF; up to 512 MB and 8 hours) or subtitles. A transcription provider turns it into a Chinese–English transcript from which questions can be generated.

Record a live class	A class is transcribed as it happens: the browser captures microphone or shared-tab audio, the platform recognizes speech locally with SenseVoice or through a Gemini Live connection and translates sentence by sentence. The recording survives disconnects and can be continued.
Convert a large PDF	A large textbook (up to 800 MB) is converted on the learner's own computer by MinerU and imported as one paged book that can be chosen by chapter. Nothing is uploaded.
Search a large textbook	Above 150,000 characters, or on request, the selection is narrowed to the pages a retrieval provider finds (a search extension reached through MCP, another plugin, or none).
Review book	A course is turned into a live book: explanations, practice, questions and answers, a table of contents, and a whole-period study plan that hands off to the final-week plan in one reading surface.
Learning flows	A flow teaches a topic in short lessons followed by practice. Flows can be edited and run as sessions.
Daily plan	The platform proposes a plan from real due reviews, weak areas and exam dates. Actions become tasks only after the learner accepts.
Knowledge outline	An outline groups topics, checks coverage and can be drawn as a diagram by the main chat assistant with an external tool.
Hand off to the main chat	Where the built-in way does not fit, the learner sends the situation to the DSH main chat, which then uses StudyHub's tools.
Guidance	The platform lists the guidance it has shown, keeps a daily budget for proactive tips and lets the learner turn them off or bring them back.

4. Requirements and quality attributes

4.1 Goals and constraints

- **Goals:** questions that cite their page; the learner's material stays local unless a feature that sends it is used; cost shown before it is spent; works with whichever model the learner has; practice that does not depend on being online.
- **Constraints:** runs as a DSH plugin (DSH 0.2.x) on Cordis; shares the host's React 18 instance; Windows x64 and macOS on Apple silicon, or the web version on Node.js 22.19+; a small team, so built-in readers rather than heavy dependencies; MIT-licensed and free.

4.2 Operating envelope

- A single learner per library, with libraries the size of a real one: about 1,000 sources (4.8 million characters, 15 courses), 58 decks, about 2,900 cards, about 150 practice runs and about 1,100 attempts, about 32 MB of stored shards. The panel must stay responsive at that size and the host's memory must not keep growing with use.
- Reading, practising, rating and scheduling must not need a model. An operation that does need one declares it and, without a model, answers "unavailable" instead of failing later.
- The expected tokens of a generation run are shown before it starts and the actual tokens after it ends.
- A background job has an execution budget (by default 20 minutes from the moment its queue slot opens, adjustable from 5 to 180) and a bound of 10 minutes per phase. By default a part holds at most 5 questions and 4 parts run in parallel. Stopping a job is safe to repeat and never loses approved work.
- Importing is bounded per format: PDF 8 MB and 200 pages, Word and PowerPoint 40 MB, text formats 8 MB, JSON deck 2 MB, audio 512 MB and 8 hours, and a PDF converted locally up to 800 MB.
- The product must survive a host restart at any time without corrupting the library, and a library written by an older version must stay readable.

- The interface works in Chinese and English; background work keeps the language it started with.

4.3 Security and privacy requirements

- The library stays on the learner's computer. Source text leaves it only when the learner uses a feature that sends it to the chosen provider (generation, help, transcription); "suggest a focus" sends only titles and headings.
- Transcription keys are stored outside the library, in the DSH home, and never enter exports or backups. Model keys are held by DSH.
- Source text is given to the model as evidence, never as instructions to follow.
- Previews remove scripts, event handlers, embedded browsing, forms and external loading. No operation reads a path that the caller supplies at read time.
- Statistics about the product's own use are off by default, never sent anywhere and not readable by the assistant.

4.4 Quality attribute scenarios, ranked

#	Attribute	Scenario (stimulus, response, measure)	Priority
Q1	Trust (correctness)	A generated question's quote is on the cited page (checked in code); a question whose review fails is not published; the learner sees why. No unreviewed question is published.	Critical
Q2	Data integrity	A host restart or a stop at any moment leaves the library valid; two windows editing one draft cannot overwrite each other; an older library stays readable and is backed up before migration.	Critical
Q3	Cost control	The expected tokens are shown before a run; a job stops at its budget (by default 20 minutes in total, 10 per phase); there is no unbounded retry loop.	High
Q4	Responsiveness	On a library of about 1,000 sources and 2,900 cards, a click plus its snapshot takes about 100 ms of server time and the panel's main thread stays under about 65 ms per changed poll.	High
Q5	Recoverability	A job interrupted by a restart leaves its approved questions as a draft marked interrupted and can be continued without rewriting what passed review.	High
Q6	Privacy	Keys are outside the library and its backups; source text is sent only through the chosen feature; usage statistics are off by default.	High
Q7	Extensibility	A new capability is added as a context or plugin without editing existing contexts; unloading removes it cleanly.	High
Q8	Portability	The same package runs in the desktop app and in the web version, on both operating systems; a missing host capability degrades with a clear message.	Medium
Q9	Usability	A newcomer reaches a first deck in about five clicks without a manual; guidance can be turned off.	Medium
Q10	Operability	Every background job is visible in one console with its progress, calls, tokens and events; failures say their cause in plain words.	Medium

Why trust outranks speed and features. A study tool that is fast but wrong is worse than none: a wrong question teaches a wrong answer and the learner cannot tell. That is why the design pays for an independent review call per part, grounds every quote in code, and puts a draft stage the learner must read in front of publication, even though each of these costs tokens and time.

5. Architecture overview

5.1 Style and principles

- **A modular monolith inside the host process**, not microservices: one learner, one machine, and DSH as the only process boundary.

- **Contexts with enforced boundaries.** Each owns its records, declares its dependencies and reaches others only through granted operations. ESLint and `tests/architecture-boundaries.test.mjs` check actual imports.
- **Reads are free; model work is declared.** An operation that needs a model says so and degrades.
- **Every write is a transaction** with a version check, an idempotency key (`operationId`) and a receipt.
- **One shape for every background job**, so the console, compact cards and agents never learn kind-specific formats.
- **Pure shared modules, thin workers.** Quote matching, quality checks, the FSRS-6 scheduler, batching and messages are pure functions that tests can exercise without a model.

5.2 Layered view

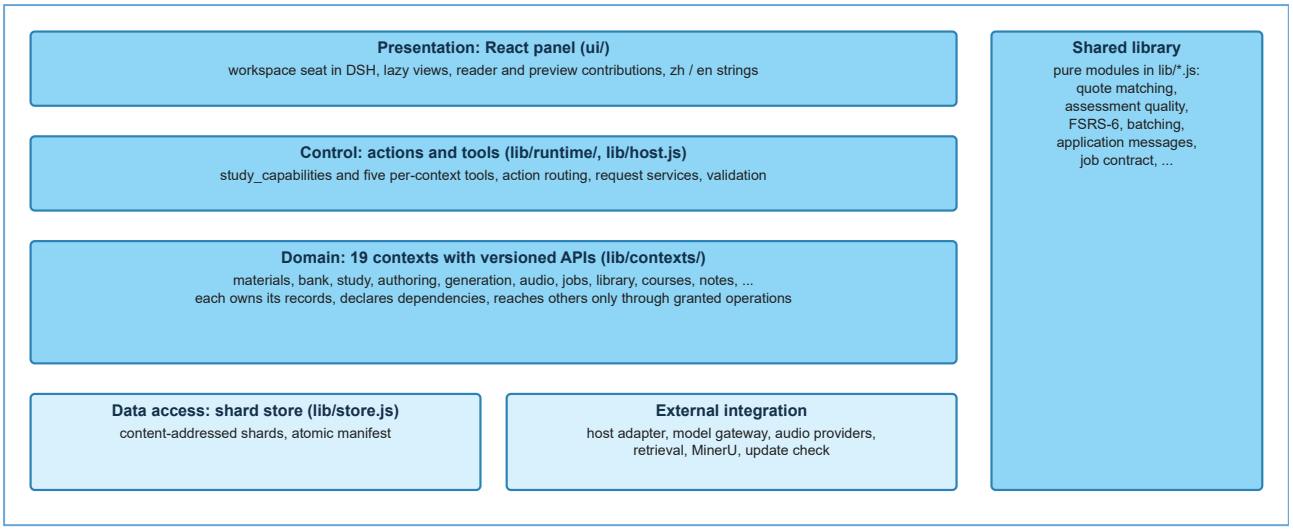


Figure 2: Architecture layers

The presentation layer is the React panel inside DSH. The control layer routes actions and tools, validates input and supplies request services (model route, language, cancellation signal, notices). The domain layer holds the contexts, each with a versioned API and its own records. The data access layer is the shard store. The external integration layer holds the host adapter, the model gateway, the audio providers, retrieval, the local converter and the update check. The shared library layer hosts pure modules.

5.3 Logical view

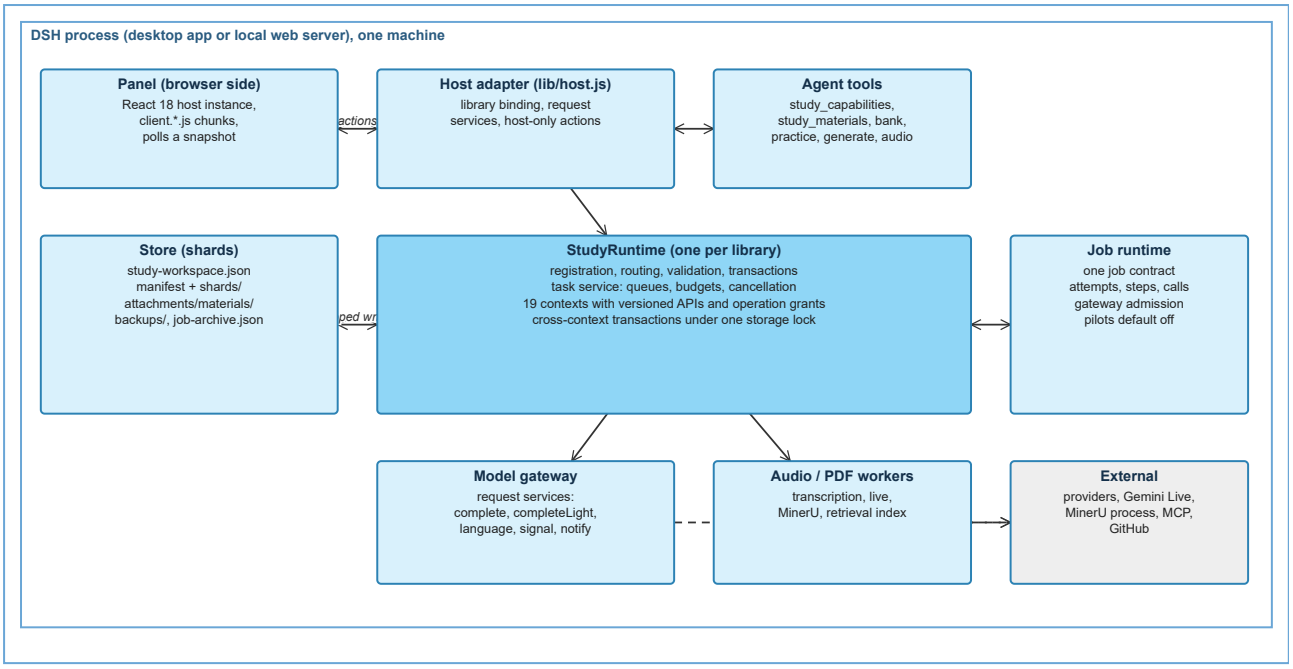


Figure 3: Logical architecture

The panel runs on the browser side of DSH and polls a snapshot. Everything else runs in the DSH process on the same machine: the host adapter, the agent tools, one `StudyRuntime` per library (with its task service and the job runtime), the store, the model gateway and the workers that call external services.

5.4 Deployment view

StudyHub is delivered as an npm package (a tarball) that DSH's plugin manager installs; there is no StudyHub server. It runs wherever DSH runs: the desktop app, or a web version served from the learner's machine (Node.js 22.19 or later).

5.5 Domain model and contexts

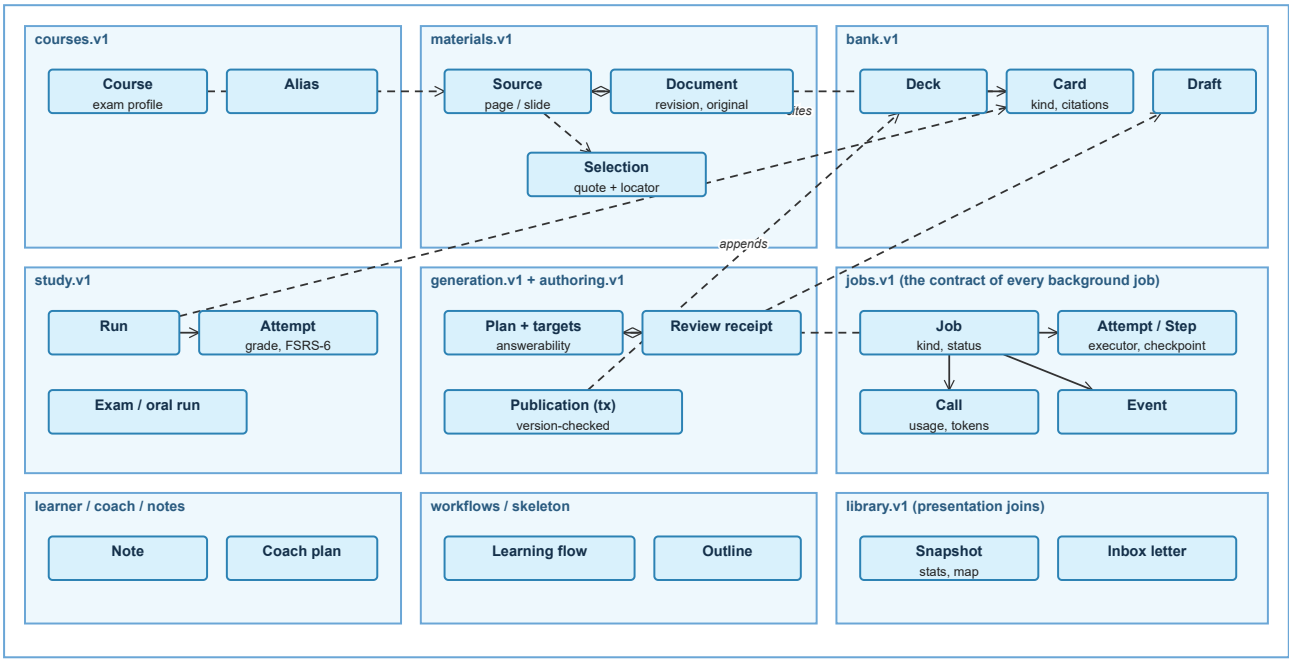


Figure 4: Domain model (main entities and owning contexts)

Group	Contexts (API)	Responsibility
Content	materials.v1, bank.v1	Sources, documents and revisions, originals, exact selections and links; decks, drafts and cards with idempotent appends
Practice	study.v1, coach.v1, learner.v1, workflows.v1, workflow-data.v1	Runs, grading, attempts, scheduling, exams and oral exams; daily plan and coaching; learning flows and their sessions
Making	generation.v1, authoring.v1	Generation, supplement and repair jobs with independent review, and the course book, outline and exam-point builds; publication as a transaction across content and study records
Capture	audio.v1, recording.v1	Transcription, live class, MinerU conversion; conversation capture
Structure	courses.v1, skeleton.v1, notes.v1, skeleton-data.v1	Courses with stable ids, outlines and topic groups, study notes
Operations	jobs.v1, library.v1, system.v1, notifications.v1	Public job status and control; snapshots, backup and restore, presentation joins; settings; inbox

There are 19 contexts in all. Three (learner, workflow-data and skeleton-data) are data-only: they own records and expose only a state read. The tools an agent sees are `study_bank`, `study_materials`, `study_generate_selection`, `study_practice`, `study_audio`, `study_capabilities`, `study_workspace`; the last refuses the usage record and Settings-only actions by name.

5.6 Packaging of a key use case

The diagram shows how the functional elements of “generate and publish a deck” are packaged into subsystems and where each one runs.

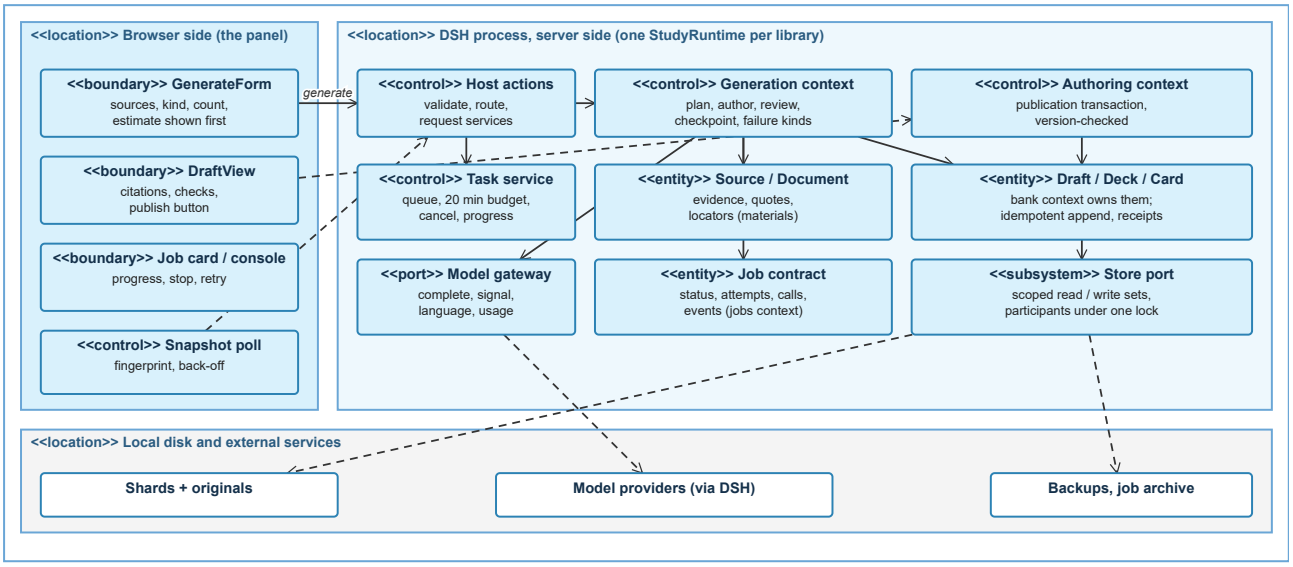


Figure 5: Packaging and deployment of “generate and publish a deck”

6. Key design decisions

ADR-01 A family of DSH plugins on one shared runtime, not a standalone application or a service	
Context	The learner's sources and recordings are private; model keys already live in DSH; a hosted service would add cost, accounts and a privacy burden to a free tool.
Options	(a) A standalone desktop app; (b) a hosted SaaS; (c) a plugin family for the host the learner already uses.
Decision	(c). 7 plugins (the workbench and 6 capability plugins) share one `studyRuntime` service keyed by a global symbol, so standalone and bundled installs never register a context twice.
Evidence	docs/architecture.md (plugin entries, full install, runtime service); package exports.
Trade-offs	No handling of model keys, no server to run and a smaller privacy surface, at the price of coupling to DSH: the supported range is DSH 0.2.x, the window belongs to the host, and host limits become product limits. A case seen in practice: in the desktop app's window a learner's tab-audio source in the live class was refused within milliseconds, with no picker. The live class now classifies such a refusal, says what it means on the learner's system, and offers the microphone (preselected after the refusal) and, on Windows, a loopback recording device instead.

ADR-02 One StudyRuntime hosting 19 contexts with versioned APIs and operation grants	
Context	A fast-growing feature set (sources, generation, practice, audio, notes, flows, outlines, jobs) needs boundaries so that one change does not ripple everywhere.
Options	(a) One large service object; (b) microservices; (c) in-process contexts with enforced boundaries.
Decision	(c). Each context declares owned fields, reads, exact operation grants and per-action write sets (`lib/runtime/domain-contracts.js`); cross-context transactions use named participants under one storage lock; the dependency graph is acyclic.
Evidence	The architecture-boundaries test and ESLint on `lib/contexts/**` check real imports; the earlier `StudyService` remains only as a compatibility facade.
Trade-offs	The boundaries cost ceremony (contracts, grants, projections), and some shared pure modules still sit outside any context. In return, contexts can be loaded and unloaded independently, and what enforces the boundary is a test, not a convention.

ADR-03 A local, content-addressed shard store with an atomic manifest, not a database	
Context	The library must be portable, inspectable, backed up as plain files, and installable without a native database in a plugin.
Options	(a) SQLite; (b) one big JSON file; (c) JSON shards named by content hash plus a small manifest that is replaced atomically.
Decision	(c). Each source, deck, draft and run is a shard; attempts are chunked; a commit writes only changed shards and then replaces the manifest atomically; a damaged library becomes write-protected; a migration keeps a backup. The store is at version 5.
Evidence	lib/store.js; docs/architecture.md (Storage); scoped read and write projections measured in docs/performance.md.
Trade-offs	Simple, diffable and portable, with natural deduplication and cheap backups. It forced work to avoid copying the whole library on each click (section 8), keeps the parsed library in memory, and has no query engine. Whole-library memory use is the main scaling limit (section 13).

ADR-04 Reads never need a model; model-dependent operations declare it and degrade	
Context	Practice must work offline and cost nothing; an operation that needs a model must not fail halfway because a key is missing.
Options	(a) Check at call time and throw; (b) declare `requiresModel` in the operation contract and answer `{ available: false, reason: "model_unavailable" }`.
Decision	(b). The UI shows a setup step instead of a failing step, and agents see availability through `study_capabilities`.
Evidence	docs/architecture.md (Models); the model-setup gate in the interface.
Trade-offs	Every operation carries a flag that must be right. In return browsing, answering and scheduling cost exactly nothing, and failure comes before work, not after it.

ADR-05 Grounded generation with one independent review, bounded repair and checkpointed drafts	
Context	Trust (Q1) and cost (Q3) pull in opposite directions: more checking costs more tokens and time.
Options	(a) One call that writes questions; (b) write and review in a loop until the count is reached; (c) plan, settle answers, write, review once independently, repair a bounded amount, checkpoint, and let the learner review.
Decision	(c). A part (at most 5 questions) goes through: *planning* (targets with quotes and an answerability declaration), *answer blueprint* (answers settled before writing), *authoring* (the author self-checks), *zero-cost text repair* in code, then **one independent review** (an unusable reply is re-asked at most 2 times). Cards flagged only for wording are re-worded once, in one call for the part, and re-reviewed alone. A shortfall is filled in at most 2 fill rounds by default (up to 4): replacement cards from a reserve first, newly planned targets after; a part whose review could not be read is reviewed again from its saved candidates. Adopting reviewers' optional suggestions is off by default. Accepted cards are checkpointed into a draft.
Evidence	lib/generation.js (authorStage, generateDeck, patchRound); lib/generation-settings.js; docs/generation-reliability.md and docs/generation-cancellation.md.
Trade-offs	Fewer questions than requested is a legitimate outcome and is reported as partial; the learner pays for independence (an extra call per part) and must still read the draft, because the content itself can be wrong. The loops are bounded by design: no unbounded refill to reach a count.

ADR-06 One job contract, and a migration to one job runtime	
Context	Audio import, PDF conversion, question runs, translation, repair, publication checks and the course book and outline builds each had their own record shape, which made the console, cards and agents kind-specific.
Options	(a) Keep per-kind records; (b) one read contract adapted from each kind; (c) a unified runtime (Job, Attempt, Step, Call) with capability declarations and managed model admission.
Decision	(b) now, (c) by migration. The contract (9 states, stage codes, progress, actions, result, usage, calls, events) is produced by one adapter. The runtime is introduced behind 24 default-off switches, each routing only new submissions of one job family, so an Attempt keeps the executor it started with.
Evidence	docs/job-contract.md; MIGRATION_SWITCHES in lib/runtime-config.js.
Trade-offs	Consumers learn one shape and the migration can be rolled back by a switch, but two worlds coexist for a long time, restart recovery exists only for some families, and some single-file formats still write unvalidated versions.

ADR-07 Idempotent, version-checked commits with receipts	
Context	The UI, agents and a restarted job can all repeat a request, and two windows can edit one draft.
Options	(a) Last writer wins; (b) pessimistic locks; (c) optimistic version checks plus an `operationId` and a stored receipt.
Decision	(c). Appends check the destination version and the evidence under the library lock, keep existing card ids and progress, and record a receipt once; replaying a committed operation returns its receipt; reusing an id with different content is rejected; publication refuses a stale version.
Evidence	docs/architecture.md (commit and recovery rules).
Trade-offs	Callers must carry versions and ids. In return a retry is always safe and silent overwrites do not happen.

ADR-08 Retrieval is delegated through one contract	
Context	Large textbooks exceed what can be sent to a model; building and maintaining an index inside the plugin would multiply native dependencies.
Options	(a) Build an index in StudyHub; (b) one `retrieve()` contract with three providers (an MCP tool, another plugin's service, or none).
Decision	(b), with a companion extension installed from the release after a SHA-256 check. Narrowing applies above 150,000 characters or on request.
Evidence	docs/architecture.md (Retrieval providers); lib/large-documents.js.
Trade-offs	A smaller core and replaceable search, but quality depends on the provider, and installing the extension has its own supply-chain steps.

ADR-09 A fingerprinted snapshot that the panel polls, with structural sharing and back-off	
Context	The host offers request and response; the panel needs a consistent view of a library that background jobs keep changing.
Options	(a) Push events; (b) poll a full snapshot; (c) poll a fingerprinted snapshot, ship only changed keys and back off when idle.
Decision	(c). An unchanged poll answers with the fingerprint only; the snapshot carries a 160-character excerpt and the length instead of source text; the poll runs every 2.5 seconds while anything moves and slows to 5 seconds after 30 seconds and 10 seconds after 2 minutes without change. Since 4.7.0 the progress of a running job is outside the fingerprint: while a job is active the panel reads only the active job rows (`job.live.list`, panel only) after each unchanged poll, and the whole snapshot is fetched again only when something structural changes (a job starts, pauses, ends or writes a draft).
Evidence	docs/performance.md (what changed); ui/poll-schedule.js; lib/job-structure.js and ui/app/live-jobs.js (live job rows).
Trade-offs	Simple and robust across host versions, with a bounded cost per poll. Latency is a polling interval, and the host still builds the snapshot on each change. Push would be better where the host allows it.

ADR-10 Convert large PDFs locally with MinerU; no cloud conversion	
Context	Large textbooks are exactly what learners cannot or will not upload; a cloud route added tokens, quotas and a privacy question.
Options	(a) A cloud conversion service; (b) a local converter installed with one click; (c) an external tool the learner runs.
Decision	(b), with (c) as a documented alternative; the cloud route was removed.
Evidence	docs/mineru-conversion.md; README (Large textbooks).
Trade-offs	Privacy and zero token cost against disk space, install time and the machine's own CPU or GPU. The install is kept safe by running in one folder, off the system drive by default, without admin rights.

ADR-11 Treat source text as evidence, never as instructions	
Context	A learner's PDF can contain text that looks like instructions to a model.
Options	(a) Trust the text; (b) present source text as labelled evidence and validate every output in code.
Decision	(b). Prompts present sources as evidence; outputs are validated structurally (JSON, ids, kinds), their quotes are checked against the page, and nothing from a source is executed or followed as a command.
Evidence	README (How generation checks questions); lib/assessment-quality.js and lib/quote-match.js.
Trade-offs	This reduces but cannot remove prompt-injection risk (a model can still be misled into writing a wrong question); the learner's review of the draft is the last control.

ADR-12 No telemetry by default; an opt-in local usage record	
Context	Improving a product needs knowledge of how it is used, but this product's promise is that material stays local.
Options	(a) Remote telemetry; (b) none; (c) an opt-in, local-only record that the learner can read and the assistant cannot.
Decision	(c): off by default, never sent anywhere, and refused by name at the assistant's tool boundary (lib/assistant-boundary.js).
Evidence	README (Data and privacy); docs/architecture.md (agent tools).
Trade-offs	Slower feedback loops (the project relies on learner reports and tests), and the strongest privacy position.

7. Critical flows

7.1 Document to deck

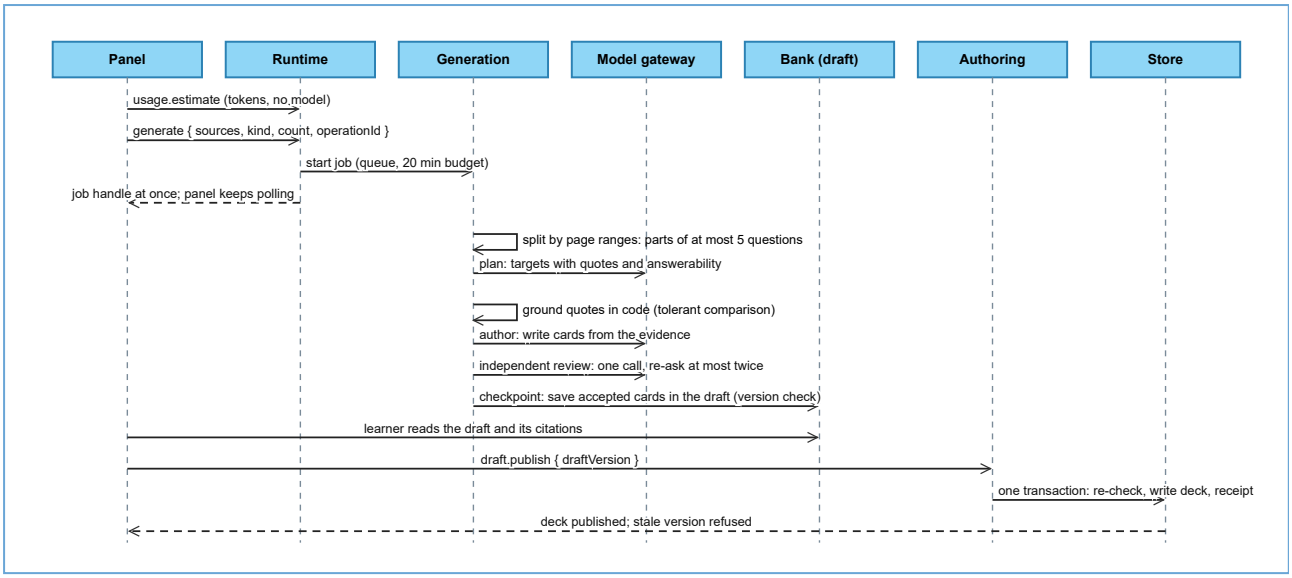


Figure 6: Generate and publish a deck (estimate, job, checkpoints, publication)

- The estimate is a pure computation. The job returns a handle at once and the panel keeps polling.
- A big selection is split by page ranges: a group is a run of whole pages of at most 60,000 characters asked for at most 10 questions, and a part holds at most 5, so a single stubborn quote cannot sink a dozen questions.
- Each part is checkpointed into the draft with a version check, so an interruption keeps what passed review, and a second window editing the draft stops the checkpoint instead of being overwritten.
- Publication re-checks every question in one transaction and refuses a stale version.

7.2 Stop, interruption and continuation

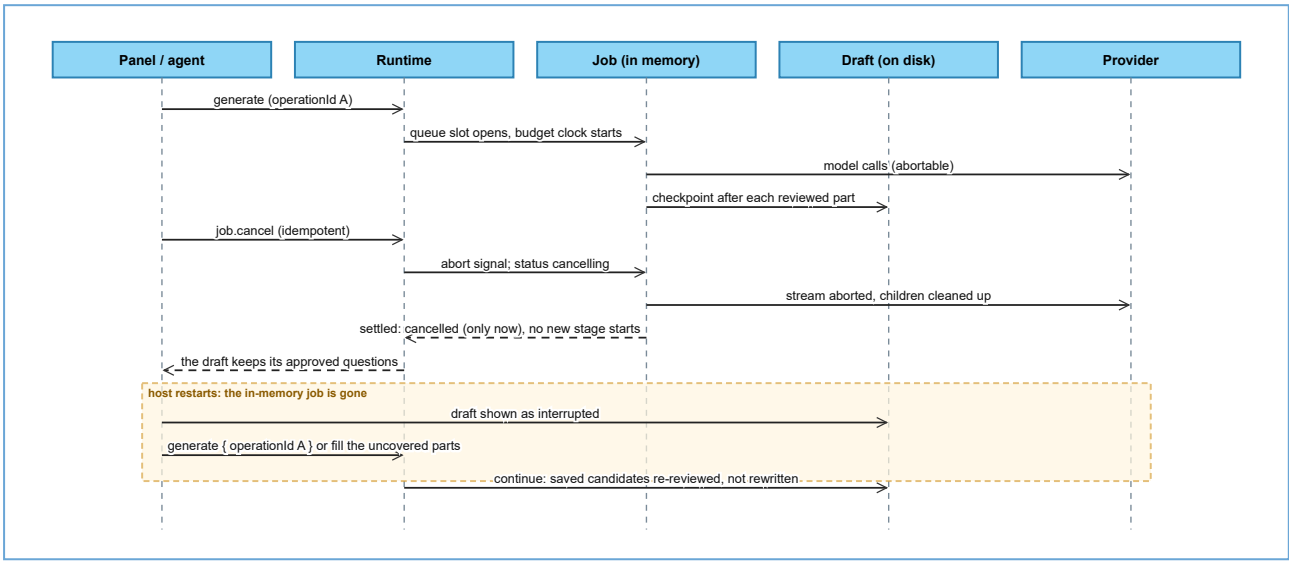


Figure 7: Stopping a job and continuing after a restart

Queued cancellation is terminal at once. Running cancellation reports *cancelling* until the provider has settled and child cleanup has finished, and the interface must not claim a worker has stopped before it has. Cancel is idempotent and scoped to its library. After a host restart the in-memory job is gone but the

saved draft is labelled interrupted; continuing under the same `operationId` re-reviews saved candidates instead of rewriting them.

7.3 Practice and scheduling (no model)

A run is built from due reviews, weak areas and unfinished work. Each answer is an attempt appended to a chunked shard, and FSRS-6 chooses the next review from the 0 to 5 rating and the days really elapsed. When the course has an exam date ahead, the next review is due no later than the exam day (a switch in the settings, on by default); only the due date moves, the memory estimate and the mastery level are computed from the unmoved gap. Only `review.reveal` and `review.answer` write, with scoped read and write sets, so after the 2.4.0 work a click costs about 25 to 100 ms of server time on a 1,000-source library (section 8).

7.4 Live class capture

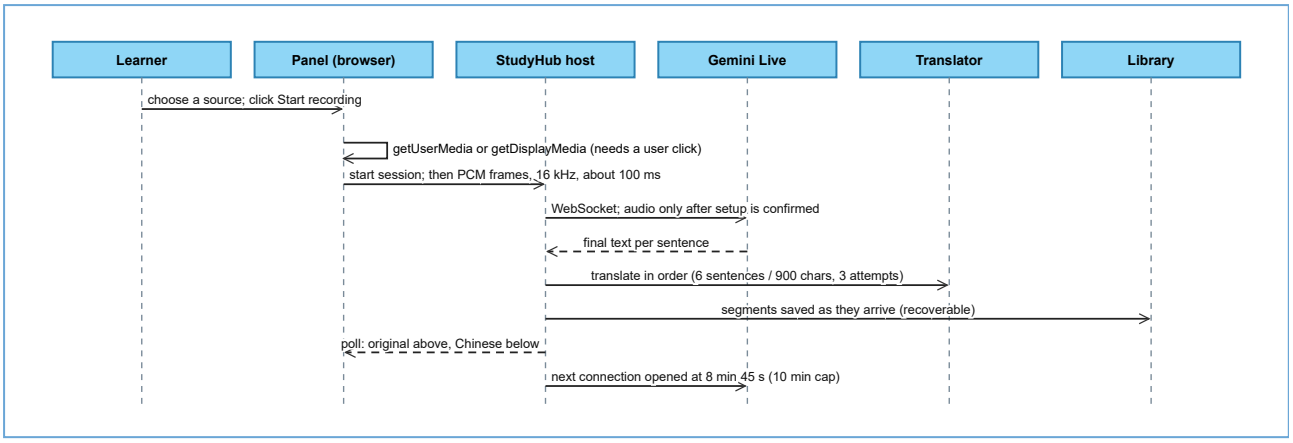


Figure 8: Live class capture, recognition and translation

Recognition runs either locally in a worker thread with SenseVoice (silero VAD and int8 model files verified by SHA-256 on install) or over Gemini Live. A Gemini Live connection is capped at 10 minutes, so the next one is opened at 8 minutes 45 seconds. After a drop the host buffers at most the last 30 seconds of audio and reconnects with increasing waits. Segments are saved as they arrive, so a disconnect loses only words that were not yet final and the class can be continued.

8. Data architecture and the performance engineering story

The durable shape is the shard store, courses with stable ids (store version 5), retained originals under `attachments/materials/<sha256>.<format>`, a job archive holding the newest 200 records of the last 90 days, and backups taken before any migration or restore. A full export includes copied originals; originals attached by reference are only listed.

8.1 What was measured (release 2.4.0)

The 2.4.0 work was driven by measurement on a copy of a real library: 1,012 sources (4.8 million characters, 15 courses), 58 decks, 2,923 cards, 147 runs, 1,091 attempts; 1,254 shard files, 31.8 MB of shards. The findings and fixes:

Dominant cost (CPU profile)	Share	Fix
`structuredClone` in state reads: every snapshot copied about 30 MB	18%	Read-only handlers read committed, deep-frozen shards without copying; transactions copy only the fields they write

`JSON.parse` of shards: a click parsed about 21 MB, then the next snapshot parsed it again	18%	Shared parsed shards keyed by content hash; shard text dropped after parsing
`JSON.stringify` in the transaction guard, to prove read-only fields untouched	18%	Frozen read-only fields skip the proof
Garbage collection caused by the above (440–713 MB of transient heap per click)	9%	Disappears with the copies
The snapshot's own build (derived facts rebuilt every time)	10%	Compute once per committed value (9,031 → 461 stringify calls)

	Before (2.4.0)	**After**
Snapshot response	8.81 MB (6.87 MB source text)	1.96 MB (no source text)
Full snapshot, median	500.6 ms	96.7 ms
review.reveal / next snapshot	179.8 ms / 578 ms	25 ms / 116 ms
Review actions per second, each followed by a poll	1.3	6.7
Cold first snapshot / peak RSS	984 ms / 323 MB	625 ms / 231 MB
Host RSS after 100 polls	700 MB	275 MB
Panel main thread per changed poll	163–213 ms, 2–4 long tasks	about 64 ms, no long task

Source: docs/performance.md; one Windows machine, Node 22. The budgets that keep these wins are pinned by tests/perf-budget.test.mjs, which passes on the current code; the figures themselves are as recorded for 2.4.0 and are not re-measured on every release.

The lesson. The slowness did not come from the domain. It came from three habits that are easy to adopt and expensive at scale: defensive copying, re-parsing what was just parsed, and proving immutability by serialising everything. The remedy was structural (frozen shared values, scoped write sets) and was protected by regression budgets so that it stays fixed.

9. Runtime, jobs and recovery

- **One task service** offers deduplication by key, FIFO queues, progress, detached results and cancellation. Scopes include the domain and the work owner, so unloading a domain aborts that domain's tasks and unloading one host cancels only its own work. The completed history keeps the latest 100 tasks.
- **Ownership of long-running work** is explicit for live sessions, uploads, assist histories, reusable teachers and panel bridges; a captured state callback cannot commit after its owner is removed, even if the same context id is registered again.
- **Writes are serialised** per library folder in-process (`lib/store-lock.js`); a file lock only arbitrates between processes. Cross-context transactions run under the same lock.
- **One writer per fact.** The usage ledger is written once per model call by the gateway, and requests answered while the learner waits are never jobs.
- **Recovery needs positive evidence.** A job is recovered only when its executor is known to be lost and the input, version and checkpoint validate; uncertain requests and artifact commits are reconciled, and intents do not count as observed calls.

> **Status.** Restart recovery today covers question runs started with the restart switch, audio manifests and interrupted PDF conversions. At a checkpoint a job can be paused only if its kind declares it (audio-import, translation, generation, coach-daily); PDF conversion, supplements, repairs, publication checks and extension tasks cannot be paused, and under the runtime pilots pause is implemented for audio and translation only. All 24 migration switches default to off, so with all of them off the behaviour is that of release 2.7.1.

10. AI architecture: cost, trust and failure

10.1 Model access

StudyHub does not hold model keys for text work. Each request gets request services from the host: `complete` and `completeLight` (the session's model route), the interface language, a cancellation signal, notices and the token meter. Audio providers have their own cards and keys, kept outside the library.

10.2 Cost control

- The estimate is shown before a run and the actual after it, and Statistics totals tokens over time.
- Budgets bound every job (by default 20 minutes in total and 10 per phase; parts of at most 5 questions, 4 in parallel), and a run that reaches its budget keeps its approved checkpoints.
- No unbounded loop: one review per part, at most 2 re-asks, one wording patch, and 2 fill rounds by default.

10.3 Failure taxonomy

Failures are classified by cause in `lib/generation-failure.js` (review-protocol, quote, plan, quality, quota, credential, budget, cancelled, no-reply, rate-limit, timeout, unavailable, else unknown). The interface describes each in plain words (title, cause, whether it was retried, a hint), and a draft carries a part report (total, passed, partial, failed, reasons), so "5 parts could not cite their source" is visible instead of a vague error. A review of this feature found that a hint must name only a button that exists on the screen that shows it.

10.4 Trust controls, in layers

Layer	Control	What it catches
Planning	Targets with quotes and an answerability declaration (mode, context available, answer only in the source list, criteria would reveal the answer)	Unanswerable or self-revealing questions before they are written
Answers	An answer blueprint settles each target's answer from the evidence before any question is written; targets without a supported answer are dropped	Questions whose answer the sources do not support
Code	Tolerant quote grounding (letters and digits after NFKC and case folding; hyphenation, markdown, tables and ligatures ignored); structure checks; topic and hint lint; zero-cost text repair	Quotes that are not on the page; answers leaked in the stem; malformed cards
Independent review	One separate call per part; an unusable reply cannot grant acceptance	Wrong answers, ambiguity, weak distractors
Learner	The draft, with citations, is read before publication; publication re-checks	Whatever the layers above can still miss

11. Packaging, delivery and quality engineering

11.1 Delivery

- One npm tarball is the full package. 6 capability archives, the search extension, English and Chinese setup pages and a checksum list make up the release assets. The installer is DSH's plugin manager, and `cordis.patch.yml` composes the full install with independently switchable components.
- React sources build into DSH client modules loaded as lazy chunks through the host's official `require.async` protocol, sharing the host's React instance (16 lazily loaded views). A standalone preview and a static demo are built from the same sources.
- The repository has 81 changelog entries between 2026-10-01 and 2026-11-10, from about 3,260 commits: one pull request per theme, one changelog fragment per change in both languages, and a

release procedure that packs, checksums and verifies.

11.2 Quality engineering

Practice	What it protects
About 8,000 fast tests plus a slower set of browser and program-starting tests	Behaviour; the fast set runs in about six minutes on one machine
An architecture-boundaries test and ESLint rules on contexts	The context graph and grants (ADR-02)
Performance budgets (`tests/perf-budget.test.mjs`) and benchmark scripts	The wins of section 8 against regression
Locale parity and guard tests (wording, “main chat” naming, hover rules)	A bilingual interface with consistent words
Sharded CI: lint once, 3 Linux and 5 Windows test shards, a verify gate	Fast, repeatable merges (about five minutes)
This document’s own freshness test (tests/case-study.test.mjs)	The numbers in this document: a change to a quoted limit or count asks for a rebuild
Independent multi-lens review of larger changes, with real-browser checks and deliberate-breakage probes	Defects that the author’s own tests missed; used on the 3.7 changes

12. Security and privacy analysis

Threat or risk	Where it shows up	Control	Residual
Prompt injection through source text	A PDF contains instructions to a model	Sources are evidence, not instructions (ADR-11); outputs validated in code; the learner reviews the draft	A misled model can still write a wrong question
Path traversal and arbitrary file read	Attaching an original; reading document bytes	Only stored, verified paths (same real path, size and hash); no operation reads a caller-supplied path at read time	Low
Script injection in previews	HTML and Markdown sources	The renderer strips scripts, handlers, embedded browsing, forms and external loading; selections are checked against source evidence	Low
Secret leakage	Keys, tokens, backups, logs	Transcription keys outside the library; model keys held by DSH; job records carry no keys or paths; the usage record is local and unreadable by the assistant	Local files are as safe as the machine
Supply chain	Installing the search extension and MinerU	The asset is checked against the release’s SHA-256 list; dependency builds need explicit confirmation; MinerU installs in one folder without admin rights	Trust in upstream releases
Data loss	A crash, a bad migration, a wrong restore	Atomic manifest, backups before migration and restore, write-protection of a damaged library, restore refused while a job is active	Disk failure (the learner’s own backups)
Cost abuse	A runaway job	Budgets, bounded retries, an estimate before the run, idempotent operations	Provider price changes
Host capability gaps	The desktop app’s window cannot share a tab	Detect and explain; a microphone or a loopback device; documented	Some features differ by host

13. Scalability analysis and a scale-out scenario

“Scalable” here does not mean many simultaneous users. It means a single learner’s library and jobs growing, the codebase and feature set growing, and, as a question for the future, many learners sharing

material.

Dimension	Today	Bottleneck	Lever
Library size	About 1,000 sources, 2,900 cards, 32 MB of shards; host heap about 58 MB after the first snapshot and about 60 MB after 200 actions (2.4.0 record)	The parsed library is held in memory (once, after the 2.4.0 work); a snapshot is built per change	Lazy collections per context; paging for sources; keep source text out of the snapshot (done); a bounded cache
Activity rate	About 6.7 review actions per second, each followed by a poll	Snapshot build and the poll interval	Change notification instead of polling; an incremental snapshot
Background jobs	FIFO queues; 4 parts in parallel by default; a 20-minute budget	Provider rate limits and latency; the host's sub-agent capacity	Admission by purpose and budget in the managed gateway; a queue per provider
Input size	PDF 8 MB and 200 pages; big books through MinerU and retrieval	Context windows and cost	Page-range splitting; retrieval narrowing above 150,000 characters
Code and features	19 contexts; about 810 files in lib and about 1,040 in the interface; 16 lazy views	Cognitive load; guidance for newcomers	Enforced boundaries; the guidance registry and budget; small files
Team	A pull request per theme with independent review	Review capacity	Automated guards (architecture, wording, performance, this document's numbers)

13.1 Scenario: from one learner to shared libraries

Suppose learners want to share decks and keep one library in sync across devices or a class. What the current design gives, and what it would take:

Option	Idea	Fit	Verdict
A. A synced folder	Put the library in a cloud-synced folder	The manifest is replaced atomically and shards are named by content, but two writers still race on the manifest	Unsafe without a lock; warn against it
B. A shard-level sync service	A thin service stores shards as blobs by hash and the manifest with compare-and-swap on a version	Content-addressed shards are naturally idempotent and diffable; attempts are append-only chunks; decks are append-mostly with version checks; conflicts are rare and explicit (reuse the stale-version refusal)	**Recommended path**: a small service with no domain logic server-side, and shards encrypted client-side to preserve privacy
C. A multi-tenant cloud rewrite	Move contexts to a server with a database	Gives sharing and analytics but loses local-first, adds accounts, cost and privacy exposure, and rewrites the proven store	Rejected for now

What would have to change in option B: a sync cursor per library, a merge rule per collection (append-only attempts merge; decks merge by card id and version; drafts stay local), identity and access control for shared decks, and rate and cost accounting on the service. Jobs and keys stay local. The architecture's idempotency keys, receipts and version checks are exactly the primitives that sync needs.

14. Assessment: strengths, weaknesses, risks and recommended next steps

14.1 Strengths (with evidence)

- Trust is engineered in layers, and the failure reasons are visible (section 10).
- The library is safe: atomic, versioned, backed up, restorable and readable across versions (sections 4 to 8).
- Performance is measured and budgeted, with regression tests (section 8).
- Boundaries are enforced by tests, so growth has not turned into a tangle (ADR-02).
- Cost and privacy are explicit product properties, not afterthoughts (ADR-04, ADR-12).

14.2 Weaknesses and technical debt (assessment)

#	Weakness	Why it matters	Direction
W1	The whole library is held in memory, and the snapshot is rebuilt per change	A library an order of magnitude larger would strain memory and latency	Lazy and paged collections; an incremental snapshot
W2	Polling instead of change notification	Latency equals the interval; there is background cost when idle	Use host events where available; keep polling as the fallback
W3	Two job worlds coexist (legacy and runtime) behind 24 switches	Recovery and pause differ by family; more states to test	Finish the migration family by family; delete the old path
W4	Strong coupling to host capabilities	A host limit becomes a product limit (for example no tab sharing in the desktop app)	Detect, explain and offer a route; track host releases
W5	A very large feature surface for newcomers	People explore for a long time	The guidance work: one next step, tips with a budget, hover explanations
W6	A long fast-test run (about six minutes; one file takes about half of it)	It slows feedback and review rounds	Split the heaviest file; shard locally
W7	Content correctness cannot be proven	A wrong but well-cited question can pass	Keep the learner in the loop; invest in answer-checking signals and feedback
W8	Drift between docs and code, and between Chinese and English	Trust in the documentation erodes (checking this document against the code found several such drifts)	Generate numbers from code; guard tests for pairs and for this document

14.3 Risks

Risk	Likelihood / impact	Response
A DSH release outside the supported range breaks an API	Medium / High	Pin the range; plugin-composition and client-boundary tests; follow DSH release candidates
A provider changes limits, formats or pricing	High / Medium	Providers behind ports; estimates and budgets; clear failure causes
A storage format change loses data	Low / Critical	Atomic writes, backups before migration, write-protection, older-library tests
A large library hits memory limits	Medium / Medium	The W1 direction; performance budgets as an early warning
Knowledge concentrated in one maintainer	Medium / High	Docs, architecture tests, plans in the repository, review practice

14.4 Recommended next steps

1. **Bound memory and make snapshots incremental** (W1, W2): the cheapest large lever before any scale-out.
2. **Finish the job-runtime migration** (W3): one runtime that owns pause, recovery and admission for every family, then remove the legacy path.
3. **Prototype the shard-level sync service** (section 13.1, option B) behind a flag, to learn the real conflict rate before committing to sharing features.

Appendix A: Glossary

Term	Meaning
DSH	DeepSeek Harness: the AI agent application (desktop app or local web server) that hosts StudyHub, holds model keys and routes, and provides sessions, tools, sidebar and previews.
Context	A unit of the runtime that owns its records, exposes a versioned API (for example `bank.v1`) and reaches other contexts only through granted operations.
Library	The learner's data: sources, decks, answers and schedules, stored as shards in a folder on the learner's computer.
Shard	One file of the library named by a hash of its content. A commit writes only the shards that changed and then replaces the manifest atomically.
Source / document	Source: a page-sized unit of text (a PDF page, a slide, a section). Document: the file it came from, with its revisions and retained original.
Deck / draft / card	A deck holds published questions (cards); a draft holds generated questions awaiting the learner's review and publication.
Job, attempt, step, call	The shape of every background task: a job has attempts, an attempt has steps, a step has model calls; one contract describes them all.
Request services	Per-call context supplied by the host: the model route, interface language, cancellation signal, notices and token meter. Never stored.
FSRS-6	The spaced-repetition scheduling algorithm (Free Spaced Repetition Scheduler, version 6) that picks the next review date from a 0 to 5 rating and the days really elapsed since the last review.
MinerU / MCP	MinerU: a PDF converter that runs on the learner's computer. MCP: the Model Context Protocol, through which a search extension is reached.
ADR	Architecture decision record: the context, options, decision and trade-offs of one design decision.

Appendix B: Sources

Statements in this document come from the repository at the version on the cover. Where a number is read from a constant, the constant is the source; where the text describes behaviour, the module named below is.

Topic	Where to look
Architecture and storage	docs/architecture.md; lib/store.js; lib/runtime/builtins.js; lib/runtime/domain-contracts.js
Generation pipeline and failure causes	docs/generation-reliability.md; docs/generation-cancellation.md; lib/generation.js; lib/generation-settings.js; lib/generation-failure.js
Jobs and the runtime migration	docs/job-contract.md; lib/runtime-config.js; lib/jobs/contract-schema.js; lib/runtime/tasks.js
Performance measurements (release 2.4.0)	docs/performance.md; tests/perf-budget.test.mjs

Large documents, conversion and retrieval	docs/mineru-conversion.md; lib/large-documents.js; README (Large textbooks)
Limits, defaults and counts quoted in the text	Read from the code when the document is built (see Document control); listed in docs/case-study/README.md
Product behaviour and privacy statements	README and README.zh-CN; lib/assistant-boundary.js; the interface sources in ui/